/* ⽤Streams打印7个随机数
整个过程没声明变量，declarative programming */

public class StreamDemo {

public static void main(String[] args) {

new Random(47).ints(1, 100) //1.创建IntStream对象

. distinct()

. limit(7)

. sorted()

. forEach(System.out::println);

}

}

IntStream Random.ints(int randomNumberOrigin, int randomNumberBound) {
return StreamSupport.intStream(
new RandomIntsSpliterator(this, 0L, Long.MAX_VALUE,
randomNumberOrigin, randomNumberBound),
false

);
}

StreamSupport.intStream(Spliterator.OfInt spliterator, boolean parallel) {
return new IntPipeline.Head<>(spliterator,
StreamOpFlag.fromCharacteristics(spliterator),
parallel);
}

AbstractPipeline(Spliterator<?> source, int sourceFlags, boolean parallel) {
this.previousStage = null;
this.sourceSpliterator = source;
this.sourceStage = this;
this.sourceOrOpFlags = sourceFlags & StreamOpFlag.STREAM_MASK;
this.combinedFlags = (~(sourceOrOpFlags << 1)) & StreamOpFlag.INITIAL_OPS_VALUE;
this.depth = 0;
this.parallel = parallel;
}

IntStream IntPipeline.limit(long maxSize) {
return SliceOps.makeInt(this, 0, maxSize);
}

IntStream SliceOps.makeInt(AbstractPipeline<?, Integer, ?> upstream, long skip, long limit) {
return new IntPipeline.StatefulOp<Integer>(upstream, StreamShape.INT_VALUE, flags(limit)) {
Sink<Integer> opWrapSink(int flags, Sink<Integer> sink) {
return new Sink.ChainedInt<Integer>(sink) { //sink4
long n = skip;
long m = limit >= 0 ? limit : Long.MAX_VALUE;
public void begin(long size) { downstream.begin(calcSize(size, skip, m)); }
public void accept(int t) {
if (n == 0)
if (m > 0) { m-; downstream.accept(t); }
else n-;
}
public boolean cancellationRequested() { return m == 0 || downstream.cancellationRequested();}
}
}
Spliterator.OfInt unorderedSkipLimitSpliterator(…) {…}
<P_IN> Spliterator<Integer> opEvaluateParallelLazy(…) {…}
<P_IN> Node<Integer> opEvaluateParallel(…) {…}
}
}
/* 代码执⾏时，jvm动态⽣成IntConsumer实现类，创建实例赋给action
IntConsumer.accept()中执⾏System.out::println */
void IntPipeline.forEach(IntConsumer action) {
evaluate(ForEachOps.makeInt(action, false));
}

TerminalOp<Integer, Void> ForEachOps.makeInt(IntConsumer consumer, boolean ordered) {
return new ForEachOp.OfInt(action, ordered); //feop
}
ForEachOp.OfInt(IntConsumer consumer, boolean ordered) {
super(ordered); //this.ordered = ordered
this.consumer = consumer;
}

<R> R AbstractPipeline.evaluate(TerminalOp<E_OUT, R> terminalOp) {
linkedOrConsumed = true;
return isParallel() ? terminalOp.evaluateParallel(this, sourceSpliterator(terminalOp.getOpFlags()))
: terminalOp.evaluateSequential(this, sourceSpliterator(terminalOp.getOpFlags()));
}

<S> Void ForEachOp.evaluateSequential(PipelineHelper<T> helper,Spliterator<S> spliterator) {
return helper.wrapAndCopyInto(this, spliterator).get();
}

<P_IN, S extends Sink<E_OUT>> S AbstractPipeline.wrapAndCopyInto(S sink,Spliterator<P_IN> spliterator) {

copyInto(wrapSink(Objects.requireNonNull(sink)), spliterator);

return sink;

}

<P_IN> Sink<P_IN> AbstractPipeline.wrapSink(Sink<E_OUT> sink) {
for (@SuppressWarnings("rawtypes") AbstractPipeline p=AbstractPipeline.this; p.depth > 0; p=p.previousStage) {
sink = p.opWrapSink(p.previousStage.combinedFlags, sink);
}
return (Sink<P_IN>) sink;
}

<P_IN> void copyInto() - copyIntoWithCancel(Sink<P_IN> wrappedSink, Spliterator<P_IN> spliterator) {
AbstractPipeline p = AbstractPipeline.this;
while (p.depth > 0) p = p.previousStage; //1.找到Head节点
wrappedSink.begin(spliterator.getExactSizeIfKnown()); //2.按创建顺序初始化所有sinks
p.forEachWithCancel(spliterator, wrappedSink); //3.启动流⽔线：
wrappedSink.end();
}

void IntPipeline.forEachWithCancel(Spliterator<Integer> spliterator, Sink<Integer> sink) {
Spliterator.OfInt spl = adapt(spliterator);
IntConsumer adaptedSink = adapt(sink);
do { } while (!sink.cancellationRequested() && spl.tryAdvance(adaptedSink));
} //internal iteration：⽆需programmer⾃⼰写loop逻辑

boolean RandomIntsSpliterator.tryAdvance(IntConsumer consumer) {
long i = index, f = fence; //index为0，fence为Long.MAX_VALUE
if (i < f) {
//1.⽣成下⼀个随机数
consumer.accept(rng.internalNextInt(origin, bound)); //2.串⾏执⾏⼀系列action
index = i + 1;
return true;
}
return false;
}

IntStream IntPipeline.distinct() {
return boxed().distinct().mapToInt(i - i);
}

Stream<Integer> boxed() {
return mapToObj(Integer::valueOf);
}

<U> Stream<U> mapToObj(IntFunction<? extends U> mapper) {
return new ReferencePipeline.StatelessOp<Integer, U>(this, StreamShape.INT_VALUE, StreamOpFlag.NOT_SORTED | StreamOpFlag.NOT_DISTINCT) {
@Override
Sink<Integer> opWrapSink(int flags, Sink<U> sink) {
return new Sink.ChainedInt<U>(sink) { //sink1
@Override
public void accept(int t) { downstream.accept(mapper.apply(t));}
}
}
};
}

IntStream IntPipeline.sorted() {
return SortedOps.makeInt(this);
}

Stream<P_OUT> distinct() {
return DistinctOps.makeRef(this);
}

IntStream ReferencePipeline.mapToInt(ToIntFunction<? super P_OUT> mapper) {
return new IntPipeline.StatelessOp<Integer, U>(this, StreamShape.REFERENCE, StreamOpFlag.NOT_SORTED | StreamOpFlag.NOT_DISTINCT) {
@Override
Sink<P_OUT> opWrapSink(int flags, Sink<Integer> sink) {
return Sink.ChainedReference<P_OUT, Integer>(sink) { //sink3
@Override
public void accept(P_OUT u) { downstream.accept(mapper.applyAsInt(u)); }
}
}
};
}

<T> IntStream SortedOps.makeInt(AbstractPipeline<?, Integer, ?> upstream) {
return new OfInt(upstream);
}

class OfInt extends IntPipeline.StatefulOp<Integer> {
public Sink<Integer> opWrapSink(int flags, Sink<Integer> sink) {
if (StreamOpFlag.SORTED.isKnown(flags))
else if (StreamOpFlag.SIZED.isKnown(flags))
else return new IntSortingSink(sink); //sink5
}
OfInt(AbstractPipeline<?, Integer, ?> upstream) {}
public <P_IN> Node<Integer> opEvaluateParallel(…) {…}
}

SortedOps$IntSortingSink.accept(int i) - void SpinedBuffer$OfInt.accept(int i) {
preAccept();
curChunk[elementIndex++] = i; //暂存
}

IntPipeline$Head //ints()⽣成
- IntPipeline$4 //distinct()⽣成
- DistinctOps$1 //distinct()⽣成
- ReferencePipeline$4 //distinct()⽣成
- SliceOps$2 //limit()⽣成
- SortedOps$OfInt //sorted()⽣成
- ForEachOp$OfInt //forEach()⽣成

<T> ReferencePipeline<T, T> DistinctOps.makeRef(AbstractPipeline<?, T, ?> upstream) {
return new ReferencePipeline.StatefulOp<T, T>(upstream, StreamShape.REFERENCE,StreamOpFlag.IS_DISTINCT | StreamOpFlag.NOT_SIZED) {
@Override
Sink<T> opWrapSink(int flags, Sink<T> sink) {
if (StreamOpFlag.DISTINCT.isKnown(flags)) {…}
else if (StreamOpFlag.SORTED.isKnown(flags)) {…}
else {
return new Sink.ChainedReference<T, T>(sink) { //sink2
Set<T> seen;
public void begin(long size) { seen = new HashSet<>(); downstream.begin(-1);}
public void end() { seen = null; downstream.end();}
public void accept(T t) {
if (!seen.contains(t)) {
seen.add(t); downstream.accept(t);
}
}
};
}
}
<P_IN> Node<T> reduce(…) {…}
@Override <P_IN> Node<T> opEvaluateParallel(…) {…}
@Override <P_IN> Spliterator<T> opEvaluateParallelLazy(…) {…}
};
}

sink1.begin() - sink2.begin() - sink3.begin() - sink4.begin() - sink5.begin()

sink1.end()
- sink2.end()
- sink3.end()
- sink4.end()
- sink5.end()

sink1.cancellationRequested()
- sink2.cancellationRequested()
- sink3.cancellationRequested()
- sink4.cancellationRequested() //循环次数达到limit值时，返回true，结束循环
- sink5.cancellationRequested()

sink1.accept()
- sink2.accept() //阻挡重复值
- sink3.accept()
- sink4.accept() //limit减1，传给下游
- sink5.accept() //暂存到数组，最后传给下游时排序
- terminal sink.accept() //

SortedOps$IntSortingSink.begin(long size) {
b = (size > 0) ? new SpinedBuffer.OfInt((int) size) : new SpinedBuffer.OfInt();
}

SortedOps$IntSortingSink.end() {
int[] ints = b.asPrimitiveArray(); //1.读取暂存的所有ints
Arrays.sort(ints); //2.排序
downstream.begin(ints.length); //3.
if (!cancellationWasRequested) {…}
else {
//4.将所有ints⼀次性传给下游feop
for (int anInt : ints) {
if (downstream.cancellationRequested()) break;
downstream.accept(anInt);
}
}
downstream.end(); //5.关闭下游
}
}

boolean SortedOps$IntSortingSink.cancellationRequested() {
cancellationWasRequested = true;
return false;
}
}

//consumer = 动态⽣成的IntConsumer实现类的实例， accept()中执⾏System.out::println
ForEachOp$OfInt.accept(int i) {
consumer.accept(t);
}

public class StreamCreation { //1.创建streams
public static void main(String[] args) {
//⽅法1：Stream.of()
Stream.of(new Bubble(1), new Bubble(2), new Bubble(3)).forEach(System.out::println);
Stream.of("It's ", "a ", "wonderful ", "day ", "for ", "pie!").forEach(System.out::print);
System.out.println();
Stream.of(3.14159, 2.718, 1.618).forEach(System.out::println);

//⽅法2：Collection.stream()
List<Bubble> bubbles = Arrays.asList(new Bubble(1), new Bubble(2), new Bubble(3));
System.out.println(bubbles.stream().mapToInt(b - b.i).sum());

Set<String> w = new HashSet<>(Arrays.asList("It's a wonderful day for pie!".split(" ")));
w.stream().map(x - x + " ").forEach(System.out::print);
System.out.println();

Map<String, Double> m = new HashMap<>();
m.put("pi", 3.14159);
m.put("e", 2.718);
m.put("phi", 1.618);
m.entrySet().stream().map(e - e.getKey() + ": " + e.getValue()).forEach(System.out::println);
}
}

Stream.of() - Arrays.stream() - stream(T[] array, int startInclusive, int endExclusive) {
return StreamSupport.stream(spliterator(array, startInclusive, endExclusive), false);
}

L0
LINENUMBER 13 L0
ICONST_3
ANEWARRAY Bubble //1.创建数组，⻓度3，元素类型Bubble
DUP
//2.添加数组元素
ICONST_0
NEW Bubble
DUP
ICONST_1
INVOKESPECIAL Bubble.<init> (I)V
AASTORE
DUP

…

//3.创建stream
INVOKESTATIC Stream.of ([LObject;)LStream; (itf)
GETSTATIC System.out : LPrintStream;
DUP

INVOKEVIRTUAL Object.getClass ()LClass;
POP

//4.合成lambda内部类，实现forEach()参数接⼝，创建实例
INVOKEDYNAMIC accept(LPrintStream;)Lfunction/Consumer; [
LambdaMetafactory.metafactory(…)LCallSite;
(LObject;)V,
PrintStream.println(LObject;)V,
(LBubble;)V
]
//5.forEach()
INVOKEINTERFACE Stream.forEach (Lfunction/Consumer;)V (itf)

<T> Spliterator<T> spliterator(T[] array, int startInclusive, int endExclusive) {
return Spliterators.spliterator(array, startInclusive, endExclusive,Spliterator.ORDERED | Spliterator.IMMUTABLE);
}

Spliterators.spliterator(Object[] array, int fromIndex, int toIndex,int additionalCharacteristics) {
return new ArraySpliterator<>(array, fromIndex, toIndex, additionalCharacteristics);
}

<T> Stream<T> StreamSupport.stream(Spliterator<T> spliterator, boolean parallel) {
return new ReferencePipeline.Head<>(spliterator,
StreamOpFlag.fromCharacteristics(spliterator),
parallel);
}

AbstractPipeline(Spliterator<?> source, int sourceFlags, boolean parallel) {
this.previousStage = null;
this.sourceSpliterator = source;
this.sourceStage = this;
this.sourceOrOpFlags = sourceFlags & StreamOpFlag.STREAM_MASK;
this.combinedFlags = (~(sourceOrOpFlags << 1)) & StreamOpFlag.INITIAL_OPS_VALUE;
this.depth = 0;
this.parallel = parallel;
}

L4
LINENUMBER 19 L4
ICONST_3
ANEWARRAY Bubble
DUP
ICONST_0
NEW Bubble
DUP
ICONST_1
INVOKESPECIAL Bubble.<init> (I)V
AASTORE

…

INVOKESTATIC Arrays.asList ([LObject;)LList; //数组- List
ASTORE 1 //本地变量bubbles指向数组
L5
LINENUMBER 20 L5

GETSTATIC System.out : Ljava/io/PrintStream;
ALOAD 1

INVOKEINTERFACE List.stream ()Lstream/Stream; (itf) //创建stream
INVOKEDYNAMIC applyAsInt()Lfunction/ToIntFunction; [
LambdaMetafactory.metafactory()LCallSite;
(LObject;)I,
StreamCreation.lambda$main$0(LBubble;)I,
(LBubble;)I
]
INVOKEINTERFACE Stream.mapToInt (Lfunction/ToIntFunction;)Lstream/IntStream; (itf)
INVOKEINTERFACE IntStream.sum ()I (itf)
INVOKEVIRTUAL java/io/PrintStream.println (I)V

LOCALVARIABLE bubbles Ljava/util/List; L5 L15 1

default Stream<E> Collection.stream() {
return StreamSupport.stream(spliterator(), false);
}

default Spliterator<E> spliterator() {
return Spliterators.spliterator(this, 0);
}

Spliterators.spliterator(Collection<? extends T> c, int characteristics) {
return new IteratorSpliterator<>(Objects.requireNonNull(c),characteristics);
}

<T> Stream<T> StreamSupport.stream(Spliterator<T> spliterator, boolean parallel) {
return new ReferencePipeline.Head<>(spliterator,
StreamOpFlag.fromCharacteristics(spliterator),
parallel);
}

AbstractPipeline(Spliterator<?> source, int sourceFlags, boolean parallel) {
this.previousStage = null;
this.sourceSpliterator = source;
this.sourceStage = this;
this.sourceOrOpFlags = sourceFlags & StreamOpFlag.STREAM_MASK;
this.combinedFlags = (~(sourceOrOpFlags << 1)) & StreamOpFlag.INITIAL_OPS_VALUE;
this.depth = 0;
this.parallel = parallel;
}

public class RandomWords implements Supplier<String> { //⽅法3：Stream.generate()
List<String> words = new ArrayList<>();
Random rand = new Random(47);
RandomWords(String fname) throws IOException {
List<String> lines = Files.readAllLines(Paths.get(fname)); //读取⽂件内容
for(String line : lines.subList(1, lines.size())) { //skip the 1st line
for(String word : line.split("[.?,]+")) words.add(word.toLowerCase()); //保存剩余所有words
}
}
public String get() { return words.get(rand.nextInt(words.size())); }
public String toString() { return words.stream().collect(Collectors.joining(" ")); }

public static void main(String[] args) throws Exception {
System.out.println(Stream.generate(new RandomWords("…"))
. limit(10)
. collect(Collectors.joining(" ")));
}
}

/* Returns an infinite sequential unordered stream where each element is generated by the provided Supplier.
This is suitable for generating constant streams, streams of random elements, etc. */
<T> Stream<T> generate(Supplier<T> s) {
return StreamSupport.stream(new StreamSpliterators.InfiniteSupplyingSpliterator.OfRef<>(Long.MAX_VALUE, s),
false);
}

Collector<CharSequence, ?, String> joining(CharSequence delimiter) {
return joining(delimiter, "", "");
}
Collector<CharSequence, ?, String> joining(CharSequence delimiter,CharSequence prefix,CharSequence suffix) {
return new CollectorImpl<>(() - new StringJoiner(delimiter, prefix, suffix),
StringJoiner::add, StringJoiner::merge,StringJoiner::toString, CH_NOID);
}

Stream<P_OUT> limit(long maxSize) {
return SliceOps.makeRef(this, 0, maxSize);
}

<T> Stream<T> SliceOps.makeRef(AbstractPipeline<?, T, ?> upstream,long skip, long limit) {
return new ReferencePipeline.StatefulOp<T, T>(upstream, StreamShape.REFERENCE,flags(limit)) {
Sink<T> opWrapSink(int flags, Sink<T> sink) { //sink1
return new Sink.ChainedReference<T, T>(sink) {
long n = skip;
long m = limit >= 0 ? limit : Long.MAX_VALUE; //10
public void begin(long size) { downstream.begin(calcSize(size, skip, m)); }
public void accept(T t) {
if (n == 0)
if (m > 0) { m-; downstream.accept(t); }
else n-;
}
public boolean cancellationRequested() { return m == 0 || downstream.cancellationRequested();}
}
<P_IN> Node<T> opEvaluateParallel(…) {…}
<P_IN> Spliterator<T> opEvaluateParallelLazy() {…}
Spliterator<T> unorderedSkipLimitSpliterator() {…}
}
}

<R, A> R ReferencePipeline.collect(Collector<? super P_OUT, A, R> collector) {
A container = evaluate(ReduceOps.makeRef(collector));
return collector.characteristics().contains(Collector.Characteristics.IDENTITY_FINISH)
? (R) container
: collector.finisher().apply(container);
}

<T, I> TerminalOp<T, I> ReduceOps.makeRef(Collector<? super T, I, ?> collector) {
Supplier<I> supplier = Objects.requireNonNull(collector).supplier();
BiConsumer<I, ? super T> accumulator = collector.accumulator(); //StringJoiner::add
BinaryOperator<I> combiner = collector.combiner();
class ReducingSink extends Box<I> implements AccumulatingSink<T, I, ReducingSink> {
public void begin(long size) { state = supplier.get(); } //StringJoiner实例
public void accept(T t) { accumulator.accept(state, t); }
public void combine(ReducingSink other) { state = combiner.apply(state, other.state); }
}
return new ReduceOp<T, I, ReducingSink>(StreamShape.REFERENCE) {
public ReducingSink makeSink() { return new ReducingSink(); } //sink2
public int getOpFlags() {…}
}
}

<R> R AbstractPipeline.evaluate(TerminalOp<E_OUT, R> terminalOp) {
linkedOrConsumed = true;
return isParallel() ? terminalOp.evaluateParallel(this, sourceSpliterator(terminalOp.getOpFlags()))
: terminalOp.evaluateSequential(this, sourceSpliterator(terminalOp.getOpFlags()));
}

<P_IN> R ReduceOp.evaluateSequential(PipelineHelper<T> helper,Spliterator<P_IN> spliterator) {
return helper.wrapAndCopyInto(makeSink(), spliterator).get();
}

<P_IN, S extends Sink<E_OUT>> S AbstractPipeline.wrapAndCopyInto(S sink,Spliterator<P_IN> spliterator) {

copyInto(wrapSink(Objects.requireNonNull(sink)), spliterator);

return sink;

}

<P_IN> Sink<P_IN> AbstractPipeline.wrapSink(Sink<E_OUT> sink) {
for (@SuppressWarnings("rawtypes") AbstractPipeline p=AbstractPipeline.this; p.depth > 0; p=p.previousStage) {
sink = p.opWrapSink(p.previousStage.combinedFlags, sink);
}
return (Sink<P_IN>) sink;
}

<P_IN> void copyInto() - copyIntoWithCancel(Sink<P_IN> wrappedSink, Spliterator<P_IN> spliterator) {
AbstractPipeline p = AbstractPipeline.this;
while (p.depth > 0) p = p.previousStage; //1.找到???
wrappedSink.begin(spliterator.getExactSizeIfKnown()); //2.按创建顺序初始化所有sinks
p.forEachWithCancel(spliterator, wrappedSink);
wrappedSink.end();
}

ReferencePipeline$Head //generate()⽣成
- SliceOps$1 //limit()⽣成

sink1.begin() - sink2.begin()

void ReferencePipeline.forEachWithCancel(Spliterator<P_OUT> spliterator, Sink<P_OUT> sink) {
do { } while (!sink.cancellationRequested() && spliterator.tryAdvance(sink));
}

sink1.accept() //循环次数达到limit值时，返回true，结束循环
- sink2.accept() //state.add()

StreamSpliterators.InfiniteSupplyingSpliterator.OfRef.tryAdvance(Consumer<? super T> action) {
//s是Stream.generate()的⼊参
action.accept(s.get()); //RandomWords.get()随机取⽂件中的⼀个word，交给sink1
return true;
}

public class Fibonacci { //⽅法4：Stream.iterate()
int x = 1;
Stream<Integer> numbers() {
return Stream.iterate(0, i - {
int result = x + i;
x = i;
return result;
});
}
public static void main(String[] args) {
new Fibonacci().numbers()
. skip(20) // Don't use the first 20
. limit(10) // Then take 10 of them
. forEach(System.out::println);
}
}

//producing a Stream consisting of seed, f(seed), f(f(seed)), etc.
<T> Stream<T> Stream.iterate(final T seed, final UnaryOperator<T> f) {
final Iterator<T> iterator = new Iterator<T>() { T t = (T) Streams.NONE;
public boolean hasNext() { return true; }
public T next() { return t = (t == Streams.NONE) ? seed : f.apply(t); }
};
return StreamSupport.stream(Spliterators.spliteratorUnknownSize(iterator, Spliterator.ORDERED | Spliterator.IMMUTABLE), false);
//new IteratorSpliterator<>(Objects.requireNonNull(iterator), characteristics);
}

… - IteratorSpliterator.tryAdvance(IntConsumer action) {
if (it.hasNext()) {
action.accept(it.next()); //执⾏Stream.iterate()⼊参⾥的内容，将结果交给sinks处理
return true;
}
return false;
}

public class FileToWordsBuilder { //Stream.builder()
static String path = “…/Cheese.dat”;
Stream.Builder<String> builder = Stream.builder(); //new Streams.StreamBuilderImpl<>();

public FileToWordsBuilder(String path) throws Exception {
Files.lines(Paths.get(path)) //以⽂件为data source，构建stream
. skip(1) // Skip the comment line at the beginning
. forEach(line - { for (String w : line.split("[.?,]+")) builder.add(w);});
} //读取⽂件内容，add all words to builder.buffer

Stream<String> stream() { return builder.build(); } //以buffer为data source，创建stream

public static void main(String[] args) throws Exception {
new FileToWordsBuilder(path).stream()
. limit(7)
. map(w - w + " ")
. forEach(System.out::print);
}
}

Stream<String> Files.lines(Path path) - lines(Path path, Charset cs) {
BufferedReader br = Files.newBufferedReader(path, cs);
try { return br.lines().onClose(asUncheckedRunnable(br)); }
catch (Error|RuntimeException e) {…}
}

Stream<String> BufferReader.lines() {
Iterator<String> iter = new Iterator<String>() { String nextLine = null;
public boolean hasNext() {…} //readLine()
public String next() {…} //返回nextLine
}
return StreamSupport.stream(Spliterators.spliteratorUnknownSize(iter, Spliterator.ORDERED | Spliterator.NONNULL), false);
//new ReferencePipeline.Head<>()
}

StreamBuilderImpl.add(T t) - accept(T t) {
if (count == 0) { first = t; count++; }
else if (count > 0) {
if (buffer == null) {
buffer = new SpinedBuffer<>();
buffer.accept(first);
count++;
}
buffer.accept(t); //缓存
}
}

SpinedBuffer.accept(E e) {
if (elementIndex == curChunk.length) { /* 扩容 - 存⼊数组 */ }
curChunk[elementIndex++] = e;
}

StreamBuilderImpl.build() {
int c = count;
if (c >= 0) {
count = -count - 1;
return (c < 2) ? StreamSupport.stream(this, false) : StreamSupport.stream(buffer.spliterator(), false);
//new ReferencePipeline.Head<>()
}
}

Spliterator<E> SpinedBuffer.spliterator() {
class Splitr implements Spliterator<E> {
int splSpineIndex;
final int lastSpineIndex;
int splElementIndex;
final int lastSpineElementFence;
E[] splChunk;
Splitr(…) {…}
public long estimateSize() {…}
public int characteristics() {…}
public boolean tryAdvance(Consumer<? super E> consumer) {…}
public void forEachRemaining(Consumer<? super E> consumer) {…}
public Spliterator<E> trySplit() {…}
}
return new Splitr(0, spineIndex, 0, elementIndex);
}

public class StreamOfStreams {

public static void main(String[] args) {

Stream.of(1, 2, 3) //1.⽣成stream，data source是[1, 2, 3]

//2.
. flatMap(i - Stream.of("Gonzo", "Kermit", "Beaker"))

. forEach(System.out::println);

}

}

<R> Stream<R> ReferencePipeline.flatMap(Function<? super P_OUT, ? extends Stream<? extends R>> mapper) {
return new StatelessOp<P_OUT, R>(this, StreamShape.REFERENCE,StreamOpFlag.NOT_SORTED | StreamOpFlag.NOT_DISTINCT | StreamOpFlag.NOT_SIZED) {
Sink<P_OUT> opWrapSink(int flags, Sink<R> sink) {
return new Sink.ChainedReference<P_OUT, R>(sink) {
public void begin(long size) { downstream.begin(-1); }
public void accept(P_OUT u) {
try (Stream<? extends R> result = mapper.apply(u)) { //执⾏flatMap()⼊参⾥的action，得到新的stream
if (result != null) result.sequential().forEach(downstream); //将新stream的所有objects传给下游
}
}
};
}
};
}

Stream.of() - Arrays.stream() - stream(T[] array, int startInclusive, int endExclusive) {
return StreamSupport.stream(spliterator(array, startInclusive, endExclusive), false);
}

Arrays.spliterator() - Spliterators.spliterator(Object[] array, int fromIndex, int toIndex,int additionalCharacteristics) {
return new ArraySpliterator<>(array, fromIndex, toIndex, additionalCharacteristics);
}

<T> Stream<T> StreamSupport.stream(Spliterator<T> spliterator, boolean parallel) {
return new ReferencePipeline.Head<>(spliterator,
StreamOpFlag.fromCharacteristics(spliterator),
parallel);
}

public ArraySpliterator(Object[] array, int origin, int fence, int additionalCharacteristics) {
this.array = array;
this.index = origin;
this.fence = fence;
this.characteristics = additionalCharacteristics | Spliterator.SIZED | Spliterator.SUBSIZED;
}

… - copyInto(Sink<P_IN> wrappedSink, Spliterator<P_IN> spliterator) {
if (!StreamOpFlag.SHORT_CIRCUIT.isKnown(getStreamAndOpFlags())) {
wrappedSink.begin(spliterator.getExactSizeIfKnown());
spliterator.forEachRemaining(wrappedSink);
wrappedSink.end();
}

…

}

ReferencePipeline$Head
- ReferencePipeline$7

ReferencePipeline$7$1.begin()
- ForEachOps$ForEachOp$OfRef.begin()

public void forEachRemaining(Consumer<? super T> action) {
Object[] a; int i, hi;
if ((a = array).length >= (hi = fence) &&(i = index) >= 0 && i < (index = hi)) {
//遍历ArraySpliterator.array⾥的所有objects，交给指定的consumer
do { action.accept((T)a[i]); } while (++i < hi);
}
}

public class TreeSetOfWords {
public static void main(String[] args) throws IOException {
Set<String> words = Files.lines(Paths.get("/Users/annietian/IdeaProjects/HelloJava/src/streams/TreeSetOfWords.java")) //读取⼀⾏
. flatMap(s - Arrays.stream(s.split("\\w+"))) //拆成words，⽣成新的stream，将所有words传给下游
. filter(s - !s.matches("\\d+")) //过滤数字
. collect(Collectors.toCollection(TreeSet::new)); //保存到TreeSet中
System.out.println(words);
}
}

<R, A> R ReferencePipeline.collect(Collector<? super P_OUT, A, R> collector) {
A container = evaluate(ReduceOps.makeRef(collector));
return collector.characteristics().contains(Collector.Characteristics.IDENTITY_FINISH)
? (R) container //返回TreeSet实例
: collector.finisher().apply(container);
}

Collectors.toCollection(Supplier<C> collectionFactory) {
return new CollectorImpl<>(collectionFactory, Collection<T>::add,
(r1, r2) - { r1.addAll(r2); return r1;}, CH_ID);
}

CollectorImpl(Supplier<A> supplier, BiConsumer<A, T> accumulator,
BinaryOperator<A> combiner,Set<Characteristics> characteristics) {
this.supplier = supplier; //TreeSet::new
this.accumulator = accumulator; //Collection<T>::add
this.combiner = combiner; //r1.addAll(r2); return r1;
this.finisher = finisher; // i - (R) i
this.characteristics = characteristics;
}

<T, I> TerminalOp<T, I> ReduceOps.makeRef(Collector<? super T, I, ?> collector) {
Supplier<I> supplier = Objects.requireNonNull(collector).supplier(); //TreeSet::new
BiConsumer<I, ? super T> accumulator = collector.accumulator(); //Collection<T>::add
BinaryOperator<I> combiner = collector.combiner(); //r1.addAll(r2); return r1;
class ReducingSink extends Box<I> implements AccumulatingSink<T, I, ReducingSink> {
public void begin(long size) { state = supplier.get(); } //TreeSet实例ts
public void accept(T t) { accumulator.accept(state, t); } //ts.add(t)
public void combine(ReducingSink other) { state = combiner.apply(state, other.state); }
}
return new ReduceOp<T, I, ReducingSink>(StreamShape.REFERENCE) {
public ReducingSink makeSink() { return new ReducingSink(); }
public int getOpFlags() {…}
}
}

<R> R AbstractPipeline.evaluate(TerminalOp<E_OUT, R> terminalOp) {
linkedOrConsumed = true;
return isParallel() ? terminalOp.evaluateParallel(this, sourceSpliterator(terminalOp.getOpFlags()))
: terminalOp.evaluateSequential(this, sourceSpliterator(terminalOp.getOpFlags()));
}

<P_IN> R ReduceOp.evaluateSequential(PipelineHelper<T> helper,Spliterator<P_IN> spliterator) {
return helper.wrapAndCopyInto(makeSink(), spliterator).get(); //返回TreeSet实例
}

ReferencePipeline$7$1 //flatMap()完成line转words
- ReferencePipeline$2$1 //filter()过滤数字
- ReduceOps$3ReducingSink //collect()将words存⼊TreeSet实例

//执⾏pipeline operations
… - copyInto(Sink<P_IN> wrappedSink, Spliterator<P_IN> spliterator) {
if (!StreamOpFlag.SHORT_CIRCUIT.isKnown(getStreamAndOpFlags())) {
wrappedSink.begin(spliterator.getExactSizeIfKnown());
spliterator.forEachRemaining(wrappedSink);
wrappedSink.end();
}

…

}

… - Iterator.forEachRemaining(Consumer<? super T> action) {
while (hasNext()) action.accept(next()); //iterator读取⼀⾏，交给sink
}

public class ParallelDemo {
public static void main(String[] args) throws IOException {
Stream.of(1,1, 2,2, 3,3, 4,4, 5,5, 6,6, 7,7, 8,8, 9,9)
. parallel() //设置并发执⾏标志sourceStage.parallel = true
. filter(t - t > 1)
. filter(t - t > 2)
. distinct() //stateful
. filter(t - t > 3)
. limit(3) //stateful
. filter(t - t > 4)
. forEach(System.out::println);
}
}

void ReferencePipeline.forEach(Consumer<? super E_OUT> action) {
evaluate(ForEachOps.makeRef(action, false));
}

<T> TerminalOp<T, Void> ForEachOps.makeRef(Consumer<? super T> action,boolean ordered) {
return new ForEachOp.OfRef<>(action, ordered);
}

//2.终点操作
ForEachOp.evaluateParallel(PipelineHelper<T> helper,Spliterator<S> spliterator) {
new ForEachTask<>(helper, spliterator, helper.wrapSink(this)).invoke();
}

ForEachOps$ForEachTask(PipelineHelper<T> helper,Spliterator<S> spliterator,Sink<S> sink) {
super(null);
this.sink = sink; //所有tasks共⽤同⼀条流⽔线
this.helper = helper;
this.spliterator = spliterator;
this.targetSize = 0L;
}

ForkJoinTask.invoke() {
int s;
if ((s = doInvoke() & DONE_MASK) != NORMAL) reportException(s);
return getRawResult();
}

int doInvoke() {
int s; Thread t; ForkJoinWorkerThread wt;
return (s = doExec()) < 0 //使⽤⼆分法，将数据源递归拆成⼦任务，提交ForkJoinPool公共池执⾏
? s

: ((t = Thread.currentThread()) instanceof ForkJoinWorkerThread)
? (wt = (ForkJoinWorkerThread)t).pool.awaitJoin(wt.workQueue, this, 0L)
: externalAwaitDone(); //等待任务结束
}

… - ForEachTask.compute() {
Spliterator<S> rightSplit = spliterator, leftSplit;
long sizeEstimate = rightSplit.estimateSize(), sizeThreshold;
if ((sizeThreshold = targetSize) == 0L) targetSize = sizeThreshold = AbstractTask.suggestTargetSize(sizeEstimate);
boolean isShortCircuit = StreamOpFlag.SHORT_CIRCUIT.isKnown(helper.getStreamAndOpFlags());
boolean forkRight = false;
Sink<S> taskSink = sink;
ForEachTask<S, T> task = this;
while (!isShortCircuit || !taskSink.cancellationRequested()) {
if (sizeEstimate <= sizeThreshold || (leftSplit = rightSplit.trySplit()) == null) { //1.数据源拆成两半
task.helper.copyInto(taskSink, rightSplit); //6.当前数据源只剩1个元素时，直接执⾏
break;
}
ForEachTask<S, T> leftTask = new ForEachTask<>(task, leftSplit); //2.左边⼀半创建leftTask
task.addToPendingCount(1); //3.当前task.pending + 1
ForEachTask<S, T> taskToFork;
if (forkRight) { forkRight = false; rightSplit = leftSplit; taskToFork = task; task = leftTask; }
else { forkRight = true; taskToFork = leftTask; }
taskToFork.fork(); //4.leftTask加⼊ForkJoinPool的公共池，由worker thread继续执⾏leftTask.compute()
sizeEstimate = rightSplit.estimateSize(); //5.更新当前task元素数
}
task.spliterator = null;
task.propagateCompletion(); //7.
}

Spliterator<T> ArraySpliterator.trySplit() {
int lo = index, mid = (lo + fence) >>> 1;
//数组左边⼀半给新的spliterator处理
return (lo >= mid) ? null : new ArraySpliterator<>(array, lo, index = mid, characteristics);
}

ForkJoinTask<V> fork() {
Thread t;
if ((t = Thread.currentThread()) instanceof ForkJoinWorkerThread) //当前线程是pool的worker
((ForkJoinWorkerThread)t).workQueue.push(this); //直接push到和当前worker绑定的queue⾥
else ForkJoinPool.common.externalPush(this); //push到公共池中
return this;
}

//其他数据源拆分逻辑
Spliterator<T> IteratorSpliterator.trySplit() { //CASE1.Collection：将元素拆到数组⾥，新建ArraySpliterator
Iterator<? extends T> i; long s;
if ((i = it) == null) { //Collection.stream()
i = it = collection.iterator(); //集合的iterator
s = est = (long) collection.size(); //集合元素数量
} else s = est;
if (s > 1 && i.hasNext()) { //⼀直拆到集合只剩⼀个元素
int n = batch + BATCH_UNIT; //1.计算拆分数量
if (n > s) n = (int) s;
if (n > MAX_BATCH) n = MAX_BATCH;
Object[] a = new Object[n]; int j = 0;

do { a[j] = i.next(); } while (++j < n && i.hasNext()); //2.将拆分的集合元素暂存到数组

batch = j;
if (est != Long.MAX_VALUE) est -= j;
return new ArraySpliterator<>(a, 0, j, characteristics); //3.返回新的spliterator
}
return null;
}

Spliterator<T> RandomIntsSpliterator.trySplit() { //CASE2.Random：
long i = index, m = (i + fence) >>> 1;
return (m <= i) ? null : new RandomIntsSpliterator(rng, i, index = m, origin, bound);
}

Spliterator<T> StreamSpliterators.InfiniteSupplyingSpliterator.OfRef.trySplit() { //CASE3.Supplier
if (estimate == 0) return null;
return new InfiniteSupplyingSpliterator.OfRef<>(estimate >>>= 1, s);
}

ForEachTask(ForEachTask<S, T> parent, Spliterator<S> spliterator) {
super(parent);
this.spliterator = spliterator;
this.sink = parent.sink; //所有tasks共⽤同⼀条流⽔线
this.targetSize = parent.targetSize;
this.helper = parent.helper;
}

propagateCompletion() {
CountedCompleter<?> a = this, s = a;
for (int c;;) {
if ((c = a.pending) == 0) { //⼦task都结束了
if ((a = (s = a).completer) == null) { //root task
s.quietlyComplete(); //set status - notify threads waiting to join this task
return;
}
} else if (U.compareAndSwapInt(a, PENDING, c, c - 1)) return; //this.pending - 1
}
}

int externalAwaitDone() {
//1.先看看能不能帮忙加速执⾏

int s = ((this instanceof CountedCompleter)
? ForkJoinPool.common.externalHelpComplete((CountedCompleter<?>)this, 0)
: ForkJoinPool.common.tryExternalUnpush(this) ? doExec() : 0);
if (s >= 0 && (s = status) >= 0) { //2.任务还未完成，阻塞等待
boolean interrupted = false;
do {
if (U.compareAndSwapInt(this, STATUS, s, s | SIGNAL)) { //在status中设置SIGNAL信号，表示有线程阻塞等待当前task完成
synchronized (this) {
if (status >= 0) { //任务未完成时，挂起当前线程
try { wait(0L); } catch (InterruptedException ie) { interrupted = true; }
} else notifyAll(); //任务已完成，唤醒所有等待线程
}
}
} while ((s = status) >= 0);
if (interrupted) Thread.currentThread().interrupt();
}
return s;
}

… - copyInto(Sink<P_IN> wrappedSink, Spliterator<P_IN> spliterator) {
if (!StreamOpFlag.SHORT_CIRCUIT.isKnown(getStreamAndOpFlags())) {
wrappedSink.begin(spliterator.getExactSizeIfKnown());
spliterator.forEachRemaining(wrappedSink);
wrappedSink.end();
}

…

}

<R> R evaluate(TerminalOp<E_OUT, R> terminalOp) {
return isParallel()
? terminalOp.evaluateParallel(this, sourceSpliterator(terminalOp.getOpFlags())) //1.以stareful操作为界，并发执⾏
: terminalOp.evaluateSequential(this, sourceSpliterator(terminalOp.getOpFlags()));
}

/* get the source spliterator for this pipeline stage:
* 1. for a sequential or stateless parallel pipeline: just return the source spliterator
* 2. for a stateful parallel pipeline: */
Spliterator<?> AbstractPipeline.sourceSpliterator(int terminalFlags) {
Spliterator<?> spliterator = null;
if (sourceStage.sourceSpliterator != null) { spliterator = sourceStage.sourceSpliterator; sourceStage.sourceSpliterator = null; }
if (isParallel() && sourceStage.sourceAnyStateful) {
int depth = 1;
for (AbstractPipeline u = sourceStage, p = sourceStage.nextStage, e = this; u != e; u = p, p = p.nextStage) {
int thisOpFlags = p.sourceOrOpFlags;
if (p.opIsStateful()) { //遍历所有stages，对stateful的stage xxx？
depth = 0;
if (StreamOpFlag.SHORT_CIRCUIT.isKnown(thisOpFlags)) {…}
spliterator = p.opEvaluateParallelLazy(u, spliterator); //并⾏执⾏流⽔线
thisOpFlags = spliterator.hasCharacteristics(Spliterator.SIZED)
? (thisOpFlags & ~StreamOpFlag.NOT_SIZED) | StreamOpFlag.IS_SIZED
: (thisOpFlags & ~StreamOpFlag.IS_SIZED) | StreamOpFlag.NOT_SIZED;
}
p.depth = depth++;
p.combinedFlags = StreamOpFlag.combineOpFlags(thisOpFlags, u.combinedFlags);
}
}
if (terminalFlags != 0) combinedFlags = StreamOpFlag.combineOpFlags(terminalFlags, combinedFlags);
return spliterator;
}

/* distinct > 将spliterator不断拆分，交给n个ForkJoinTasks并发执⾏，最终合并所有tasks的结果（合并过程去除⼦任务间的重复数据）
（各task执⾏路径：当前stage之前所有⽆状态stages的sinks - 当前task的sink）
* > 将结果包装成新的Spliterator，返回 */
Spliterator<T> ReferencePipeline.StatefulOp.opEvaluateParallelLazy(PipelineHelper<T> helper, Spliterator<P_IN> spliterator)
{
return reduce(helper, spliterator).spliterator(); //c.stream().spliterator();
}

<P_IN> Node<T> reduce(PipelineHelper<T> helper, Spliterator<P_IN> spliterator) {
TerminalOp<T, LinkedHashSet<T>> reduceOp = ReduceOps.<T, LinkedHashSet<T>>makeRef(LinkedHashSet::new,
LinkedHashSet::add,
LinkedHashSet::addAll);
return Nodes.node(reduceOp.evaluateParallel(helper, spliterator));
}
<T> Node<T> Nodes.node(Collection<T> c) { return new CollectionNode<>(c); }

<T, R> TerminalOp<T, R> makeRef(Supplier<R> seedFactory, BiConsumer<R, ? super T> accumulator,BiConsumer<R,R> reducer) {
class ReducingSink extends Box<R> implements AccumulatingSink<T, R, ReducingSink> {
public void begin(long size) { state = seedFactory.get(); } //state指向新建的LinkedHashSet实例s
public void accept(T t) { accumulator.accept(state, t); } //s.add(t)
public void combine(ReducingSink other) {reducer.accept(state, other.state); } //s.addAll(otherSet)
}
return new ReduceOp<T, R, ReducingSink>(StreamShape.REFERENCE) {
public ReducingSink makeSink() { return new ReducingSink(); }
}
}

<P_IN> R evaluateParallel(PipelineHelper<T> helper, Spliterator<P_IN> spliterator) {
return new ReduceTask<>(this, helper, spliterator).invoke().get(); //LinkedHashSet实例
}

… - AbstractTask.compute() {
Spliterator<P_IN> rs = spliterator, ls; //ArraySpliterator
long sizeEstimate = rs.estimateSize(); //⽬前数组元素总数
long sizeThreshold = getTargetSize(sizeEstimate); //停⽌拆分阈值
boolean forkRight = false;
K task = (K) this;
//1.不断拆分提交⼦任务
while (sizeEstimate > sizeThreshold && (ls = rs.trySplit()) != null) { //1.1 数据源分2半
K leftChild, rightChild, taskToFork;
task.leftChild = leftChild = task.makeChild(ls); //1.2 创建leftTask
task.rightChild = rightChild = task.makeChild(rs); //1.3 创建rightTask
task.setPendingCount(1);
if (forkRight) {
forkRight = false;
rs = ls;
task = leftChild;
taskToFork = rightChild;
} else {
forkRight = true;
task = rightChild;
taskToFork = leftChild;
}
taskToFork.fork(); //1.4 leftTask提交到ForkJoinPool的公共池，由worker thread执⾏
sizeEstimate = rs.estimateSize();
}
task.setLocalResult(task.doLeaf()); //2. 数据源⽆法再拆分，执⾏任务
task.tryComplete(); //3.开始⾃下往上合并结果
}

Spliterator<T> ArraySpliterator.trySplit() {
int lo = index, mid = (lo + fence) >>> 1;
//数组左边⼀半给新的spliterator处理
return (lo >= mid) ? null : new ArraySpliterator<>(array, lo, index = mid, characteristics);
}

S ReduceTask.doLeaf() { //每个⼦任务构建⾃⼰的流⽔线，sinks
return helper.wrapAndCopyInto(op.makeSink(), spliterator); //new ReducingSink()
}

AbstractPipeline.wrapAndCopyInto(S sink, Spliterator<P_IN> spliterator) {
copyInto(wrapSink(Objects.requireNonNull(sink)), spliterator);
return sink;
}

… - copyInto(Sink<P_IN> wrappedSink, Spliterator<P_IN> spliterator) {
wrappedSink.begin(spliterator.getExactSizeIfKnown()); //创建LinkedHashSet实例s
spliterator.forEachRemaining(wrappedSink); //遍历数据源的所有elements，依次经过sink链处理
wrappedSink.end();
}

… - ReduceTask(ReduceTask parent, Spliterator<P_IN> spliterator) {
this.completer = parent; //this
this.spliterator = spliterator; //拆分后的数据源
this.helper = parent.helper; //和上级共⽤
this.targetSize = parent.targetSize; //和上级⼀样
this.op = parent.op; //和上级共⽤
}

public void forEachRemaining(Consumer<? super T> action) {
Object[] a; int i, hi;
if ((a = array).length >= (hi = fence) &&(i = index) >= 0 && i < (index = hi)) {
//遍历ArraySpliterator.array⾥的所有objects，交给指定的consumer
do { action.accept((T)a[i]); } while (++i < hi);
}
}
CountedCompleter.tryComplete() {
CountedCompleter<?> a = this, s = a;
for (int c;;) {
if ((c = a.pending) == 0) { //1.叶⼦节点pending值为0 3.⽗节点所有⼦任务都完成了
a.onCompletion(s); //3.1.合并结果：⼦类实现合并逻辑，所有⼦任务的结果combine之后，存⾄⽗节点localResult
if ((a = (s = a).completer) == null) { //3.2 ⽗节点为空，说明当前节点为根节点
s.quietlyComplete(); //set status - notify threads waiting to join this task
return;
} //2.⽗节点不为空，继续向上处理⽗节点 4.继续处理⽗节点
} else if (U.compareAndSwapInt(a, PENDING, c, c - 1)) return; //3.如果⽗节点还有⼦任务未完成，将⽗节点的pending值减1，
表示⾃⼰已完成，最后⼀个⼦任务完成后会合并结果
}
}

/* limit > 将spliterator不断拆分，交给n个ForkJoinTasks并发执⾏，最终合并所有tasks的结果（root task最后截取指定⻓度）
（各task执⾏路径：当前stage之前所有⽆状态stages的sinks - 当前task的sink）
* > 将结果包装成新的Spliterator，返回 */
Spliterator<T> ReferencePipeline.StatefulOp.opEvaluateParallelLazy(PipelineHelper<T> helper, Spliterator<P_IN> spliterator) {
long size = helper.exactOutputSizeIfKnown(spliterator);
…

return new SliceTask<>(this, helper, spliterator, castingArray(), skip, limit).invoke().spliterator();
} //new Nodes.InternalNodeSpliterator.OfRef<>(this)

SliceTask(AbstractPipeline<P_OUT, P_OUT, ?> op, PipelineHelper<P_OUT> helper,
Spliterator<P_IN> spliterator, IntFunction<P_OUT[]> generator, long offset, long size) {
this.completer = null;
this.helper = helper;
this.spliterator = spliterator;
this.op = op;
this.generator = generator;
this.targetOffset = offset;
this.targetSize = size; //limit值
this.sharedResult = new AtomicReference<>(null);
}

… - compute() {
Spliterator<P_IN> rs = spliterator, ls; //ArraySpliterator
long sizeEstimate = rs.estimateSize(); //⽬前data source元素总数
long sizeThreshold = getTargetSize(sizeEstimate); //停⽌拆分阈值
boolean forkRight = false;
K task = (K) this;
AtomicReference<R> sr = sharedResult;
R result;
while ((result = sr.get()) == null) {
if (task.taskCanceled()) { result = task.getEmptyResult(); break; }
//1. 数据源分2半，⽆法再拆分时，执⾏任务，结束循环
if (sizeEstimate <= sizeThreshold || (ls = rs.trySplit()) == null) { result = task.doLeaf(); break; }
K leftChild, rightChild, taskToFork;
task.leftChild = leftChild = task.makeChild(ls); //2. 创建leftTask
task.rightChild = rightChild = task.makeChild(rs); //2. 创建rightTask
task.setPendingCount(1);
if (forkRight) { forkRight = false; rs = ls; task = leftChild; taskToFork = rightChild; }
else { forkRight = true; task = rightChild; taskToFork = leftChild; }
taskToFork.fork(); //3 leftTask提交到ForkJoinPool的公共池，由worker thread执⾏
sizeEstimate = rs.estimateSize();
}
task.setLocalResult(result);
task.tryComplete(); //4.
}

Node<P_OUT> SliceTask.doLeaf() {
if (isRoot()) { //root task
long sizeIfKnown = StreamOpFlag.SIZED.isPreserved(op.sourceOrOpFlags) ? op.exactOutputSizeIfKnown(spliterator) : -1;
final Node.Builder<P_OUT> nb = op.makeNodeBuilder(sizeIfKnown, generator);
Sink<P_OUT> opSink = op.opWrapSink(helper.getStreamAndOpFlags(), nb);
helper.copyIntoWithCancel(helper.wrapSink(opSink), spliterator);
return nb.build();
}
else { //⼦任务
//1.copyInto()：遍历当前数据源的elements，传给sink链处理，最终缓存到terminalSink.curChunk中
//2.build()：返回this
Node<P_OUT> node = helper.wrapAndCopyInto(helper.makeNodeBuilder(-1, generator), spliterator).build();
thisNodeSize = node.count();
completed = true;
spliterator = null;
return node;
}
}

wrapAndCopyInto(S sink, Spliterator<P_IN> spliterator) { //sink为SpinedNodeBuilder实例
copyInto(wrapSink(Objects.requireNonNull(sink)), spliterator);
return sink;
}

… - copyInto(Sink<P_IN> wrappedSink, Spliterator<P_IN> spliterator) {
wrappedSink.begin(spliterator.getExactSizeIfKnown()); //
spliterator.forEachRemaining(wrappedSink); //遍历当前task数据源的elements，依次经过sink链处理
wrappedSink.end(); //最终缓存到当前task的sink（SpinedNodeBuilder实例）的curChunk中
}

ReferencePipeline$2$1@836 //filter(t - t > 3)对应的sink1
- Nodes$SpinedNodeBuilder@829 //limit(3)对应的sink，各task⽣成⾃⼰的sink2

ReferencePipeline$2$1@790 //filter(t - t > 1) 对应的sink1
- ReferencePipeline$2$1@789 //filter(t - t > 2) 对应的sink2
- ReduceOps$4ReducingSink@785 //distinct() 对应的sink3

task1执⾏路径：sink1-1 - sink2-1 - sink3-1（对task1的数据源去重）
task2执⾏路径：sink1–2 - sink2-2 - sink3-2（对task2的数据源去重）
task3执⾏路径：sink1-3 - sink2-3 - sink3-3（对task3的数据源去重）
…

taskn执⾏路径：sink1-n - sink2-n - sink3-n（对task4的数据源去重）

void ReduceTask.onCompletion(CountedCompleter<?> caller) {
if (!isLeaf()) {
S leftResult = leftChild.getLocalResult();
leftResult.combine(rightChild.getLocalResult()); //合并⼦任务结果，去除左右⼦任务的重复数据
setLocalResult(leftResult); //保存当前task的最终结果
}
super.onCompletion(caller);
}

CountedCompleter.tryComplete() {
CountedCompleter<?> a = this, s = a;
for (int c;;) {
if ((c = a.pending) == 0) { //⼦任务都完成了
a.onCompletion(s); //合并结果
if ((a = (s = a).completer) == null) { //root任务
s.quietlyComplete(); //set status - notify threads waiting to join this task
return;
}
} else if (U.compareAndSwapInt(a, PENDING, c, c - 1)) return;
}
}

void SliceTask.onCompletion(CountedCompleter<?> caller) {
if (!isLeaf()) {
Node<P_OUT> result;
thisNodeSize = leftChild.thisNodeSize + rightChild.thisNodeSize;
if (canceled) {…}
else if (thisNodeSize == 0) result = getEmptyResult();
else if (leftChild.thisNodeSize == 0) result = rightChild.getLocalResult();
//1.汇总⼦任务结果
else result = Nodes.conc(op.getOutputShape(),leftChild.getLocalResult(), rightChild.getLocalResult());
setLocalResult(isRoot() ? doTruncate(result) : result); //2.root task负责截取指定⻓度返回
completed = true;
}
if (targetSize >= 0 && !isRoot() && isLeftCompleted(targetOffset + targetSize)) cancelLaterNodes();
super.onCompletion(caller);
}

Node<P_OUT> doTruncate(Node<P_OUT> input) {
long to = targetSize >= 0 ? Math.min(input.count(), targetOffset + targetSize) : thisNodeSize;
return input.truncate(targetOffset, to, generator);
}

Node<T> ConcNode.truncate(long from, long to, IntFunction<T[]> generator) {
if (from == 0 && to == count()) return this;
long leftCount = left.count();
if (from >= leftCount) return right.truncate(from - leftCount, to - leftCount, generator);
else if (to <= leftCount) return left.truncate(from, to, generator);
else return Nodes.conc(getShape(), left.truncate(from, leftCount, generator), right.truncate(0, to - leftCount, generator));
}

V invoke() {
int s;
if ((s = doInvoke() & DONE_MASK) != NORMAL) reportException(s);
return getRawResult();
}

R getLocalResult() {
if (isRoot()) {
R answer = sharedResult.get(); //截取后的ConcNode实例，存有最终结果
return (answer == null) ? getEmptyResult() : answer;
} else return super.getLocalResult();
}

兼容各种数据源 > 零散数据：Stream.of()
> 随机数：Random.ints()
> 集合：Collection.stream()
> 数组：Arrays.stream()
> ⽂件：Files.lines()
> Supplier：Stream.generate()
> seed + function：Stream.iterate()

stream执⾏过程
1.创建操作链 > 起始操作：将数据源封装成Spliterator实现类 > 构造Head节点
> 中间操作intermediate operations
> 终点操作terminal operation
2.执⾏终点操作
串⾏执⾏ > wrapSink()：组装流⽔线，连接所有操作的sinks（相当于打通pipeline）
> copyInto()：初始化sinks
启动流⽔线：内部迭代数据源 > 数据依次流过下游的sinks，进⾏加⼯ > 最终到达terminal sink
关闭流⽔线
并⾏执⾏

ForkJoinTask<V> ForkJoinTask.fork() { //提交⼦任务

Thread t;
//当前线程是pool的worker，放⼊当前worker的私有queue⾥
if ((t = Thread.currentThread()) instanceof ForkJoinWorkerThread) ((ForkJoinWorkerThread)t).workQueue.push(this);

else ForkJoinPool.common.externalPush(this); //放⼊公共池

return this;

}